République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
ECOLE SUPÉRIEURE EN INFORMATIQUE
8 Mai 1945 - Sidi-Bel-Abbès

الجمهورية الجزائرية الديمقراطية الشعبية
وزارة التعليم العالي والبحث العلمي
المدرسة العليا للإعلام الآلي
8 ماي 1945 - سيدي بلعباس

Higher School of Computer Science 08 May 1945 - Sidi Bel Abbes

Oriented Object Programming

Mini Project

ESI-SBA Event Registration System

ESI-SBA
Event Registration System

Mini project Demo link:

https://drive.google.com/file/d/1cI_fEM3ApSG2qS7arCU4zruoDq4cyfEs

Scan to see Demo

Created By:
Dr, Serhane Oussama

Objective:

This mini-project is designed to build a simplified **ESI-SBA Event Registration System** using the Java programming language, illustrating core Object-Oriented Programming (OOP) principles such as inheritance, polymorphism, and the integration of GUI interfaces. The following table describe all classes and their functionalities:

Class Name	Class Description
`Person`	Parent class that stores the common information of all persons in the system, such as `id`, `name`, and `email`.
`Student`	Represents a student who can register for events. This class inherits from `Person` and stores information such as level, department, and registered events.
`Organizer`	Represents the organizer responsible for managing events. This class inherits from `Person` and can create and manage events.
`Event`	Represents an event in the system. It stores event information such as code, title, date, time, maximum participants, assigned room, and required equipment.
`Room`	Represents a room used to host an event. It stores room number, building, and capacity, and can verify whether it can host a given event.
`Equipment`	Represents equipment required for an event, such as a projector or laptop. It stores the equipment identifier, name, and available quantity.
`Registration`	Represents the registration relation between a student and an event. It stores the registration id, date, status, and links one student to one event.

Stage One (core classes and implementation):

In the first part of this mini project, you are required to design and implement a Java SE. The objective of this phase is to practice class creation, inheritance, associations between classes, constructors, methods, and testing using `main` methods.

-At this stage, the work is limited to the **console version only**; no graphical interface is required yet.

You must create all required classes (see attached class diagram) in separate files, define the relationships between them according to the class diagram, and implement the main logic of the system.

Note: Use a package name Classes and then create all classes ad shown in the **Figure 1**

A partial test must be added in some classes, and a final global test must be provided in a main class that demonstrates the correct functioning of the whole project.

Finally the Main class is also provided so you must insure that you rename the class as class diagram mention.

- The final project structure should be as shows **Figure1.**
- After a successful execution of the project it should output as shown In **Figure2**

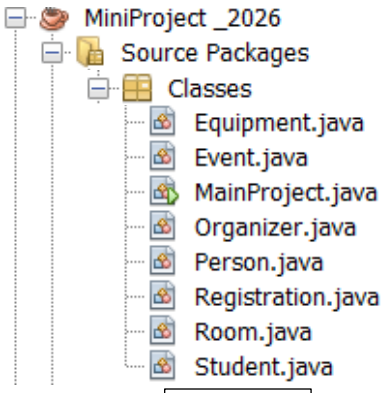


Figure1

```
run:
===== PARTIAL TESTS =====
ID: 1, Name: Ali, Email: ali@univ.dz
Level: L2, Department: Computer Science
ID: 10, Name: Sara, Email: sara@univ.dz
Club: Tech Club

===== GLOBAL TEST =====

--- Register students in Java Workshop ---
Amine registered successfully in Java Workshop
Lina registered successfully in Java Workshop
Event Java Workshop is full.

--- Try conflicting registration for Amine ---
Schedule conflict for Amine between Java Workshop and AI Seminar

--- Register Amine in non-conflicting event ---
Amine registered successfully in Robotics Session

===== EVENT DETAILS =====
Event Code: E01
Title: Java Workshop
Date: 2026-04-10
Time: 9 - 11
Max Participants: 2
Current Participants: 2
Remaining Places: 0
```

Figure2

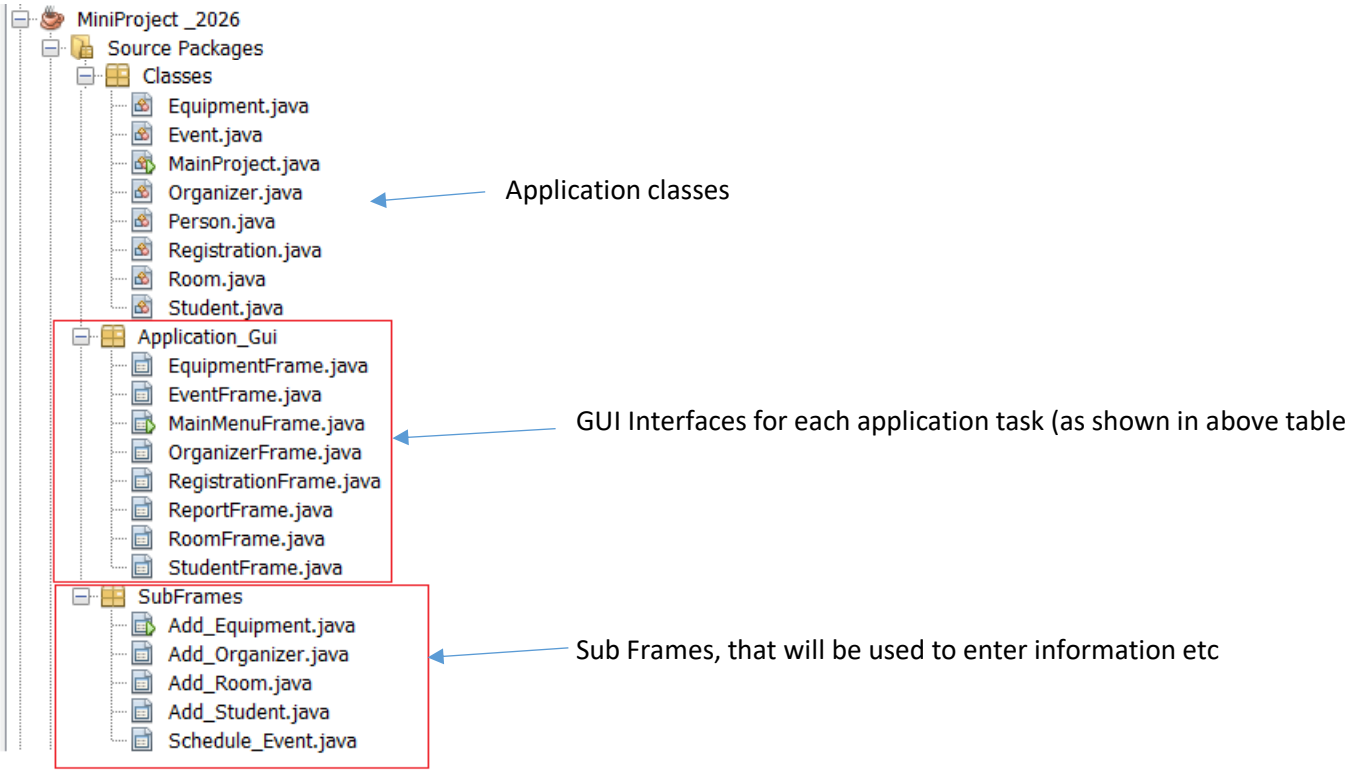
.....

Stage Two (GUI Interface using java SE)

In the second part of this mini project, you are required to design the **graphical user interface (GUI)** of the application using **NetBeans Palette / drag-and-drop tools**. The objective of this phase is to transform the console-based project into a visual desktop application. At this stage, the focus is only on the **design of the interface forms** and their organization. The interface must be created manually using NetBeans GUI Builder, and it must not be generated by AI tools. Students must build clear and functional windows that represent the main operations of the project, such as managing students, organizers, events, rooms, equipment, and registrations.

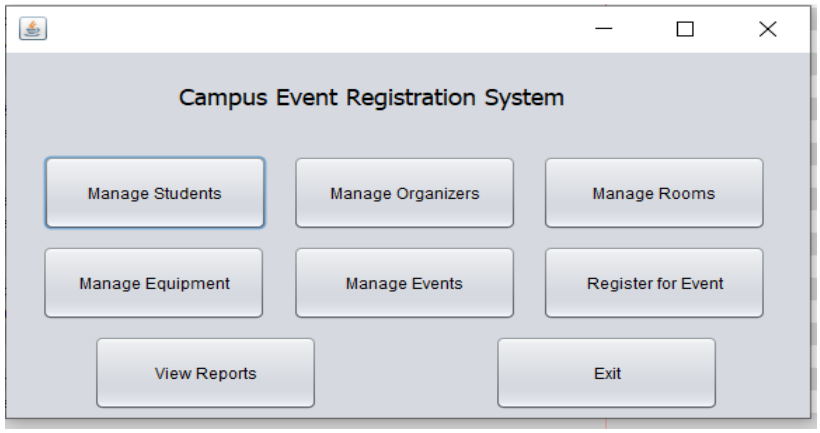
Window Name	Purpose	Main Components
`MainMenuFrame`	Main navigation window of the application	Title label, buttons to open all other windows
`StudentFrame`	Add and display students	Labels, text fields, buttons, …
`OrganizerFrame`	Add and display organizers	Labels, text fields, buttons, …
`RoomFrame`	Add and display rooms	Labels, text fields, buttons, …
`EquipmentFrame`	Add and display equipment	Labels, text fields, buttons, text area
`EventFrame`	Add and display events	Labels, text fields, combo boxes, …
`RegistrationFrame`	Register a student in an event	Combo boxes, buttons, …
`ReportFrame`	Show final information and reports	Buttons, text area or table…

First using NetBeans (drag and drop) Create two set of Frames in two different Packages namely **Application_Gui** and **SubFrames** as following:

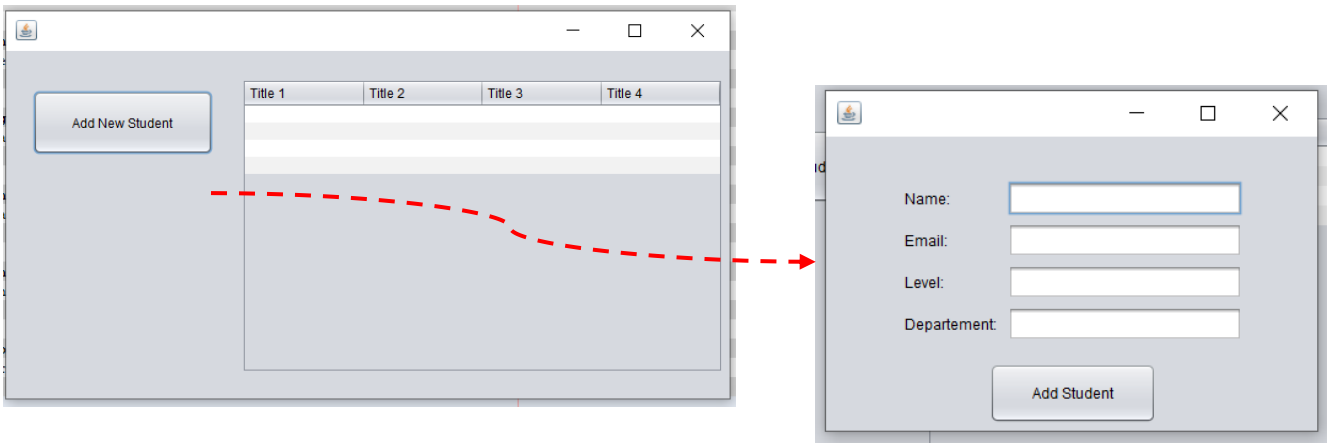


In the following, I will details the content of each frame and its workflow:

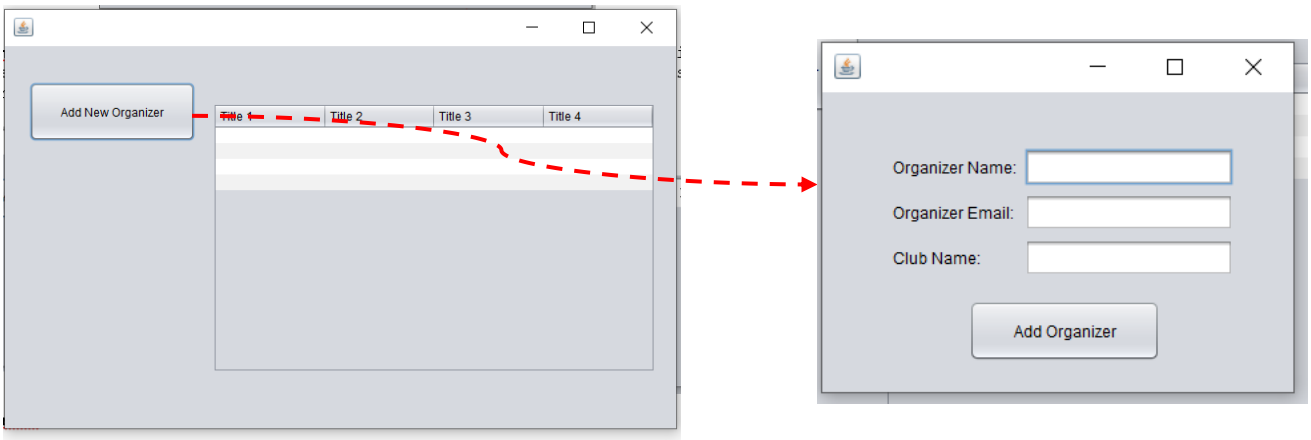
MainMenuFrame contain a set of Button that trigger (open) all the other frames



StudentFrame: this frame to display the registered (available) students. In addition, Add a new student using the Add new Student Button that open the subFrame to enter student information

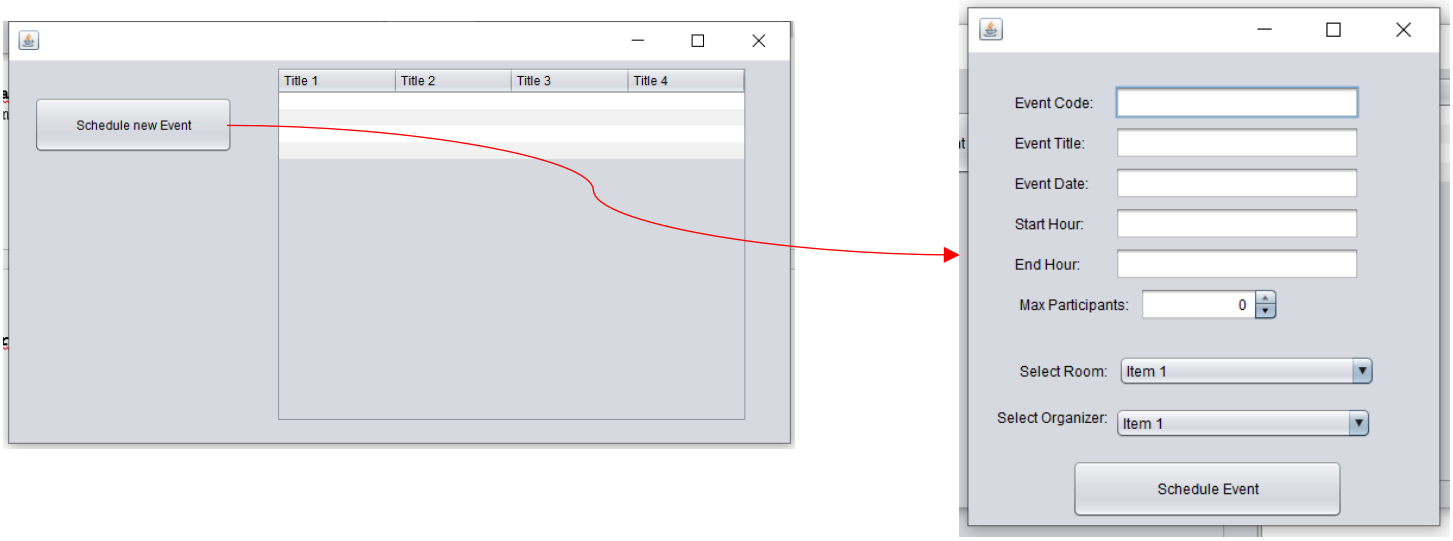


OrganizerFrame: its similar to Student Frame, its used for Organizer (add and display) the Add new Organizer open the sub Frame to enter Organizer information

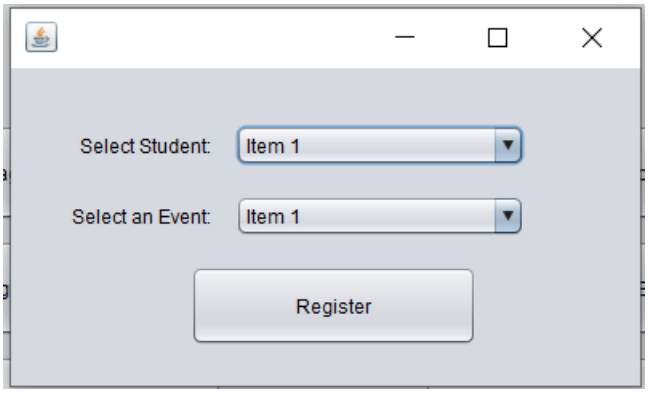


RoomFrame and **EquipmentFrame** also have similar components and functionalities (see class diagram to know what the components need for room and equipment are)

EventFrame This frame is used to create a new event, display all available events, and open a sub-frame to create additional events.



RegistrationFrame it's a simple frame used to register a student in an event



ReportFrame it's a frame that contain a table that display all events and registration information

Stage Three (Linking the GUI with Classes)

At this stage and before start implementing (the linking of classes with their interfaces) you must insure the following steps:

- First, you need start by cleaning all the frames and subFrames from their main function except the MainMenuFrame.
- Next, you need to rename all TextField and Button with meaningful variable name.
- After these to steps you need to insure the logical flow of the frames, eg (when click on Manage student button it should open StudentFrame. And in StudentFrame there is Add New Student Button, when click on it, it also should open the Add_Student subFrame etc.)
- In the following an example of all action that should be performed by the `MainMenuFrame` Buttons
  Hint: Double-click the button to access its action and insert the code to be executed or left click of the button then click on **Events** then **Action** then **Action performed …** as shown in the following **Figure 4**.

Use public access modifiers for Student attributes to allow easy access from classes in different packages.

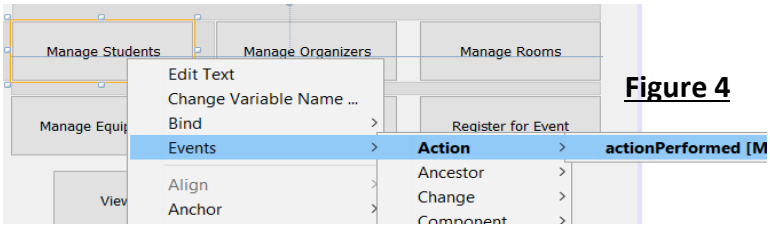


Figure 4

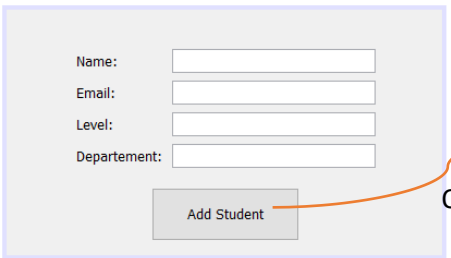
The following figure shows all actions performed when the **MainMenuFrame** buttons are clicked.

```
private void Manage_Students_BtnActionPerformed(java.awt.event.ActionEvent evt) {
    new StudentFrame().setVisible(true);
}

private void Manage_Organizers_BtnActionPerformed(java.awt.event.ActionEvent evt) {
    new OrganizerFrame().setVisible(true);
}

private void Manage_Rooms_BtnActionPerformed(java.awt.event.ActionEvent evt) {
    new RoomFrame().setVisible(true);
}

private void Register_Event_BtnActionPerformed(java.awt.event.ActionEvent evt) {
    new RegistrationFrame().setVisible(true);
}

private void Manage_Event_BtnActionPerformed(java.awt.event.ActionEvent evt) {
    new EventFrame().setVisible(true);
}

private void Manage_Equipments_BtnActionPerformed(java.awt.event.ActionEvent evt) {
    new EquipmentFrame().setVisible(true);
}

private void Report_BtnActionPerformed(java.awt.event.ActionEvent evt) {
    new ReportFrame().setVisible(true);
}

private void Exit_BtnActionPerformed(java.awt.event.ActionEvent evt) {
    System.exit(0);
}
```

The next step of this stage is to add an ArrayList for each of the following frames (before constructors). These ArrayLists will be used for rendering and storage purposes.

`StudentFrame`	public static ArrayList<Student> Students_List = new ArrayList();
`OrganizerFrame`	public static ArrayList<Organizer> Organizer_List = new ArrayList();
`RoomFrame`	public static ArrayList<Room> Room_List = new ArrayList();
`EquipmentFrame`	public static ArrayList<Registration> Registration_List = new ArrayList<>();
`EventFrame`	public static ArrayList<Event> Event_List = new ArrayList();

Now we will create a new student using the `Add_Student` sub-frame, save it in the `Students_List`, and display it in the student table in the `StudentFrame`.



On Click

```java
private void Add_Student_BtnActionPerformed(java.awt.event.ActionEvent evt) {
        // Get values from text fields
        String name = Student_Name_TextField.getText();
        String email = Student_Email_TextField.getText();
        String level = Student_Leven_TextField.getText();
        String department = Student_Departement_TextField.getText();

        // Simple validation
        if(name.isEmpty() || email.isEmpty() || level.isEmpty() || department.isEmpty()) {
            System.out.println("Please fill all fields!");
            return;
        }
        // You can generate ID automatically or set it manually
        int id = StudentFrame.Students_List.size() + 1;
        // Create Student object using input values
        Student s = new Student(id, name, email, level, department);
        // Add to list
        StudentFrame.Students_List.add(s);
        StudentFrame.loadStudentsToTable();
        System.out.println("Student added successfully");
        // Close the current frame
        this.dispose();
}
```

- The code reads student information (name, email, level, department) from text fields and checks that none are empty.
- It creates a new `Student` object with a generated ID and adds it to the `Students_List`.
- It refreshes the student table by calling `loadStudentsToTable()`, prints a success message, and closes the current window.

You need to add a `loadStudentsToTable` at `StudentFrame`:

add import for import `javax.swing.table.DefaultTableModel;`

```java
public static void loadStudentsToTable() {
// Get table model
DefaultTableModel model = (DefaultTableModel) Student_Display_Table.getModel();
// Clear existing rows
model.setRowCount(0);
    // Loop through Students_List
    for (Student s : StudentFrame.Students_List) {
        Object[] row = {
            s.id,
            s.name,
            s.email,
            s.level,
            s.department
        };
        model.addRow(row);
    }
}
```

Note that the function should be public static to be accessed directly from other classes without creating an object.

Also the Table should be a static: see a video on how to change the propriety of a table to Static:

https://youtu.be/1FhMdu8oND4

Your messing for the the next steps is to repeat (implementation logic) this process to most frames: `OrganizerFrame, RoomFrame, EventFrame.`

For `EquipmentFrame` uses a List instead of table to display the equipment. Therefore the following function is used to update the List after each addition of new equipment.

Add ➔ import javax.swing.DefaultListModel;

```
    public static void loadEquipmentToList() {
    // Create list model
    DefaultListModel<String> model = new DefaultListModel<>();
    // Loop through Equipment_List
    for (Equipment e : EquipmentFrame.Equipment_List) {
        String display = e.name + " (Qty: " + e.quantityAvailable + ")";
        model.addElement(display);
    }
    // Set model to JList
    Equipment_List_Display.setModel(model);
}
```

This function will be called after the add of a new equipment to update the List do display the new item.

Finally the `ReportFrame` contain only Table that displays all available event registration.

This mini project guide also contains a video demonstration that shows the full running example.

Important: Since you have reached here, you already notice that this not a from scratch coding. It is a simple code assembling and even most of the source code has been given.

Deadlines/Marks details will distributed as following:

Deadline	Mark/20
05-04-2026 ➔ 16-04-2026	16/20 ~ 20/20
19-04-2023 ➔ 30-04-2023	16/20 ~ 13/20
01-05-2023 ➔ 15-05-2023	13/20 ~ 10/20
15-05-2023 ➔ 20-05-2023	10/20 ~ 07/20
After 11-05-2026	00/20

Increase your mark:

1 point for the qualities/beauty of the software (eg. No error during execution, creative UI improvement …)

The following must be done in the order:

1 point for include more functionalities and clear workflow (eg. Delete, Edit, Research etc..)

1 point for transforming all the project to self executed (.jar file)